PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO DE JANEIRO
Pós-Graduação *Inteligência Artificial Generativa & Large Language Models*

CARLOS FREDERICO COELHO LISBOA

AUDITORIA DE PROJETOS POWER BI

Rio de Janeiro
2026

CARLOS FREDERICO COELHO LISBOA

AUDITORIA DE PROJETOS POWER BI

Trabalho apresentado ao curso GenAI e LLM MASTER da Pontifícia Universidade Católica do Rio de Janeiro como pré-requisito para conclusão de curso e obtenção de crédito na disciplina "Projetos de Sistemas Inteligentes de Apoio à Decisão".

Orientadora: Evelyn C. S. Batista

Rio de Janeiro

2026

LISTA DE TABELAS

Tabela 1 O sinal que cada regra lê e a forma da afirmação 10
Tabela 2 Pilha tecnológica da ferramenta 11
Tabela 3 Desempenho das regras contra o gabarito em P1–P7 (resultado oficial (tag regras--gt)) 16
Tabela 4 Desempenho das regras após a correção da MOD-003 e da PERF-001 (análise de erros) 17
Tabela 5 Recuperação vencedora (fragmento por seção, busca BM25, corpus completo): médias macro por regra em 62 consultas 18
Tabela 6 Pertinência dos trechos complementares por estratégia de busca (média macro) . . 18
Tabela 7 Rubrica humana das explicações geradas: proporção de “sim” por item 19
Tabela 8 Rubrica humana das explicações por regra 19
Tabela 9 Concordância entre a leitura humana e a conferência automática (a: ambos sim; b: só o automático diz sim; c: só o humano diz sim; d: ambos não) 20

LISTA DE FIGURAS

Figura 1 Trecho simplificado de um `model.bim` (projeto P1), com uma coluna calculada, uma medida, uma partição em Power Query M e um relacionamento 6
Figura 2 Fluxo da auditoria 11
Figura 3 Instrução de sistema enviada ao modelo de linguagem 13
Figura 4 Achado do P8 com a explicação gerada pela ferramenta (identificador da tabela abreviado) 21

SUMÁRIO

1 Método e arquitetura 5
1.1 Visão geral 5
1.2 Leitura do projeto PBIP 5
1.3 Critério de detectabilidade 8
1.4 As regras 9
1.5 Fluxo da auditoria 10
1.6 Base de documentação e recuperação 12
1.7 Geração com citação obrigatória 13
1.8 Conjunto de avaliação 14
2 Avaliação 15
2.1 Desenho da avaliação 15
2.2 Regras 15
2.3 Recuperação 17
2.4 Geração 19
2.5 Estudo de caso: o projeto P8 21
Referências 22

1 MÉTODO E ARQUITETURA

Este capítulo descreve a ferramenta desenvolvida e as decisões de método que a sustentam. A apresentação segue o caminho de um projeto auditado: a leitura do arquivo, a detecção dos problemas, a recuperação da documentação que os fundamenta e a redação da explicação pelo modelo de linguagem. As escolhas foram registradas, ao longo do projeto, em registros de decisão de arquitetura (*Architecture Decision Records*, ADR), citados aqui pelo número quando a decisão vem de um deles.

1.1 VISÃO GERAL

A ferramenta recebe um projeto do Power BI Desktop salvo no formato PBIP (*Power BI Project*) e devolve uma lista de achados, cada um com explicação, recomendação e a passagem da documentação oficial que o fundamenta. Ela se organiza em três camadas. A primeira lê o modelo semântico do projeto e o converte numa representação interna normalizada. A segunda aplica regras determinísticas sobre essa representação e produz os achados. A terceira recupera, para cada achado, trechos da documentação do Microsoft Learn e pede a um modelo de linguagem executado localmente que explique o problema e recomende a correção, citando os trechos.

A divisão de responsabilidades entre a segunda e a terceira camadas é a decisão central do projeto (ADR-003): **as regras determinísticas são a única fonte de achados**. O modelo de linguagem não decide se um problema existe; ele apenas explica um problema que uma regra já detectou. Essa separação tem duas consequências. A precisão e a revocação da auditoria passam a depender só das regras, que são reprodutíveis e podem ser medidas contra um gabarito, sem a variação própria da geração. E o modelo de linguagem, cujo erro é mais difícil de controlar, fica restrito à parte em que um erro custa menos: uma explicação ruim degrada a redação, mas não cria nem apaga um achado.

1.2 LEITURA DO PROJETO PBIP

Um relatório do Power BI tem duas partes. O *modelo semântico* reúne as tabelas, as colunas, os relacionamentos entre as tabelas e as medidas de cálculo; é a camada que os visuais consultam. O *relatório* são as páginas e os visuais construídos sobre esse modelo. No formato tradicional, o PBIX, as duas partes ficam dentro de um único arquivo binário, ilegível fora do Power BI Desktop. O formato PBIP (*Power BI Project*) grava o mesmo conteúdo como uma pasta de arquivos de texto, com uma subpasta para o relatório e outra para o modelo semântico, o que permite versionar o projeto e, para o que interessa aqui, lê-lo com um programa.

Na pasta do modelo semântico, o arquivo central é o `model.bim`. Ele contém a definição completa do modelo, escrita em TMSL (*Tabular Model Scripting Language*), a linguagem em

JSON com que a Microsoft descreve os modelos tabulares do Analysis Services, o mesmo motor que executa os modelos do Power BI (Microsoft, 2026). O arquivo guarda a **definição** do modelo, mas não os **dados**: os dados importados ficam numa cópia em cache, num arquivo separado (`cache.abf`) que o Git ignora por padrão (Microsoft, 2026). Por isso a ferramenta audita a estrutura do modelo sem acessar nem os dados nem a fonte de onde eles vêm. Nos projetos deste trabalho, o `model.bim` tem entre 53 KB e 862 KB.

A Figura 1 mostra um trecho simplificado do `model.bim` de um dos projetos avaliados. No primeiro nível ficam o nível de compatibilidade do modelo e o objeto `model`. Dentro dele, a lista `tables` descreve cada tabela: suas colunas, com tipo de dado e forma de agregação padrão, e, no caso das colunas calculadas, a expressão DAX que as define; suas medidas, cada uma com a expressão DAX; e suas partições, com a consulta em Power Query M que carrega os dados. A lista `relationships` descreve cada relacionamento pela tabela e coluna de cada ponta. O arquivo traz ainda hierarquias, papéis de segurança em nível de linha e anotações gravadas pelo Power BI Desktop.

Figura 1 – Trecho simplificado de um `model.bim` (projeto P1), com uma coluna calculada, uma medida, uma partição em Power Query M e um relacionamento

```
{
  "compatibilityLevel": 1606,
  "model": {
    "tables": [
      {
        "name": "Date",
        "columns": [
          { "name": "Date", "dataType": "dateTime", "isKey": true },
          { "name": "Mth", "type": "calculated", "dataType": "string",
            "expression": "FORMAT('Date'[Date],\"MMM\")" }
        ],
        "measures": [
          { "name": "Latest year", "expression": "min('Date'[Year])" }
        ],
        "partitions": [
          { "mode": "import",
            "source": { "type": "m",
                        "expression": ["let", "    Source = Excel.Workbook(...)", "..."] } }
        ]
      }
    ],
    "relationships": [
      { "fromTable": "Sales", "fromColumn": "CustomerKey",
        "toTable": "Customer", "toColumn": "CustomerKey" }
    ]
  }
}
```

Fonte: Elaborado pelo autor a partir do `model.bim` do projeto P1, com propriedades omitidas.

Tudo o que as regras sabem do modelo vem desse arquivo, e duas propriedades dele moldaram o método. A primeira é o que **não** está no arquivo: os valores dos dados e o uso das colunas pelos visuais do relatório. Uma regra que dependa dessas informações não pode ser decidida pela ferramenta, como mostra a próxima seção. A segunda é que o Power BI Desktop omite as propriedades que estão no valor padrão. No relacionamento da Figura 1, não aparecem a cardinalidade nem a direção do filtro porque ambas têm o valor padrão: muitos-para-um, com filtro num só sentido. Nos oito modelos do projeto, a propriedade que torna o filtro bidirecional aparece em apenas 5 dos 57 relacionamentos. Ler um `model.bim` exige, portanto, conhecer o padrão de cada propriedade, e tratar a ausência de uma propriedade como informação só é seguro quando se sabe que ela seria gravada no caso contrário. Essa lição está na origem do terceiro teste do critério de detectabilidade.

A pasta do modelo semântico declara, no arquivo `definition.pbism`, em que formato a definição está gravada: na versão 1.0, obrigatoriamente em TMSL, no `model.bim`; a partir da versão 4.0, em TMSL ou em TMDL (*Tabular Model Definition Language*), um formato mais legível que divide a definição em vários arquivos dentro de uma pasta `definition` (Microsoft, 2026). A ferramenta lê apenas o `model.bim` (ADR-001). O TMSL é JSON, que a biblioteca padrão do Python lê sem dependências, enquanto o TMDL não tinha, à época da decisão, analisador oficial em Python, e salvar em TMDL ainda era recurso em pré-visualização, de conversão irreversível. O formato fica atrás de uma única função de leitura, de modo que acrescentar o TMDL no futuro não exige reescrever as regras. Na prática, basta salvar o projeto com a opção de TMDL desligada para que o Power BI Desktop grave o `model.bim`.

O analisador converte o `model.bim` num modelo interno validado com Pydantic, com tabelas, colunas, medidas, relacionamentos, hierarquias, ordenações, papéis de segurança em nível de linha e as `variations` que ligam uma coluna de data à tabela de data automática criada pelo Power BI. Uma particularidade do TMSL é guardar expressões ora como texto, ora como lista de linhas; o analisador normaliza os dois casos.

As expressões DAX (*Data Analysis Expressions*) não são interpretadas sintaticamente, o que exigiria um analisador completo da linguagem, fora do orçamento do projeto. Elas passam por um analisador léxico próprio, que separa comentários, cadeias, números, nomes de função, referências a colunas e medidas e operadores. A afirmação de que o analisador léxico basta foi medida, não suposta: no modelo do estudo de caso (P8), as 137 expressões DAX foram transformadas em tokens sem nenhum token desconhecido, e as 105 expressões de autoria do modelador ficaram cobertas pela varredura, sem lacunas. Quando uma expressão não é coberta, a ferramenta declara a lacuna no resultado em vez de silenciar.

1.3 CRITÉRIO DE DETECTABILIDADE

O planejamento inicial previa entre 20 e 25 regras, com uma cota por categoria. A cota foi substituída, ainda no início do projeto, por um critério que pergunta outra coisa: não quantas regras cabem em cada categoria, mas quais problemas podem ser identificados de forma confiável a partir do arquivo. O critério de detectabilidade foi construído em três etapas, cada uma motivada por uma falha concreta, e é aplicado **antes** de escrever o código de uma regra.

O primeiro teste é a **decidibilidade no arquivo**: o problema precisa ser observável no `model.bim`. Ele ordena as regras em grupos de prioridade. As estruturais, que leem propriedades explícitas do JSON, vêm primeiro; as de DAX com padrão textual inequívoco, depois; as de DAX dependentes de contexto de avaliação ficaram fora, porque sem analisador sintático a heurística erra com frequência; e as de Power Query M ficaram fora do produto mínimo.

O segundo teste é a **âncora citável**: uma regra só entra se houver uma passagem do Microsoft Learn que a sustente, lida e transcrita antes do código. Sem essa passagem, a regra não pode cumprir a exigência de que toda explicação cite a documentação. Aplicado ao conjunto estrutural, o teste eliminou três das nove regras propostas. Uma delas marcaria como defeito o uso de tabelas calculadas em DAX, mas a página que a sustentaria recomenda justamente criar tabelas de data com as funções `CALENDAR` e `CALENDARAUTO` (Microsoft, 2025). As outras duas não tinham passagem que as fundamentasse.

O terceiro teste é a **suficiência da evidência**. Ele nasceu de uma regra que passava nos dois primeiros e ainda assim errava: a MOD-005, que deveria apontar a dimensão de data não marcada como tabela de data, lia a ausência de uma propriedade como prova de que a marcação não fora feita. Mas o Power BI Desktop nunca grava essa propriedade, de modo que a ausência era compatível com os dois casos e a regra marcaria também a dimensão corretamente configurada. O teste exige que a regra enumere por escrito, antes do código, (a) todas as formas de escrever a condição procurada, (b) todas as coisas que se parecem com o sinal sem sê-lo e (c) todos os lugares do arquivo onde o sinal pode estar. Só então a regra declara se afirma pela **presença** ou pela **ausência** de algo; afirmar por ausência exige as três enumerações completas e a garantia de que a presença seria gravada no caso contrário. Faltando qualquer uma, a regra se cala e o falso negativo é declarado. A MOD-005 foi reescrita: passou a usar como prova a `variations` da coluna de data apontando para uma tabela de data automática, que o Power BI remove quando a tabela é marcada. Com o tempo automático desligado, ela se cala.

O caso que melhor ilustra o critério é uma regra rejeitada. A candidata PERF-005 apontaria colunas sem uso no modelo e era a de maior rendimento esperado do grupo, com âncora já transcrita: a página de redução de dados recomenda remover colunas que não sirvam nem ao relatório nem à estrutura do modelo (Microsoft, 2024a). A cláusula (c) a derrubou. O uso de uma

coluna pelo relatório mora na camada de visuais, que a ferramenta não lê, e a regra afirmaria uma ausência sobre um domínio que não observa. A medição no P8 confirmou o problema: dos 33 achados que a regra emitiria, 5 eram defensáveis, 3 eram falsos positivos confirmados e os 25 restantes dependiam da camada de relatório. A precisão ficaria entre cerca de 15% e cerca de 91%, conforme o uso dessas 25 colunas, uma faixa larga demais para sustentar qualquer número diante do limiar de 0,7 adotado no projeto. A regra não foi implementada.

Por isso, a contagem final de regras deixou de ser meta e passou a ser **resultado** do método. O conjunto tem 9 regras, menos da metade do previsto, e a diferença não é lacuna de esforço: cada regra ausente é um caso em que o critério impediu um achado que a ferramenta não poderia sustentar. O argumento é o mesmo da literatura de análise estática, discutido no Capítulo 2: uma regra com precisão baixa é pior que uma regra ausente, porque compromete a confiança no relatório inteiro.

1.4 AS REGRAS

As 9 regras se dividem em três categorias. O catálogo completo, com a página do Microsoft Learn que ancora cada uma, está no Apêndice A.

As seis regras de **modelagem** leem a estrutura do modelo. A MOD-001 aponta o tempo automático de data/hora ligado, que cria uma tabela de data oculta para cada coluna de data e aumenta o tamanho do modelo (Microsoft, 2025). A MOD-002 aponta relacionamentos com filtro bidirecional, fonte de ambiguidade na propagação de filtros. A MOD-003 aponta tabelas sem nenhum relacionamento com o restante do modelo. A MOD-005, já discutida, aponta a dimensão de data não marcada como tabela de data. A MOD-006 aponta dimensões normalizadas em floco de neve, que o esquema estrela recomenda, em geral, integrar numa única tabela (Microsoft, 2024b). A MOD-007 aponta relacionamentos um-para-um.

As duas regras de **desempenho** leem propriedades das colunas. A PERF-001 aponta colunas calculadas em DAX: o motor VertiPaq as armazena com compressão tipicamente menos eficiente que a de colunas criadas no Power Query, e elas prolongam a atualização dos dados (Microsoft, 2024a). A PERF-003 aponta colunas de ponto flutuante cuja agregação padrão é a soma.

A regra de **DAX**, DAX-001, lê o texto das expressões e aponta divisões feitas com o operador `/` quando o denominador não é constante. A âncora mudou o enunciado da regra. A versão inicial, “divisão com `/` em vez de `DIVIDE`”, marcaria como defeito o que a própria documentação recomenda quando o denominador é constante: usar o operador, que evita um teste desnecessário (Microsoft, 2021). A regra implementada afirma apenas sobre denominador não constante, condição que as duas passagens da página sustentam juntas.

A Tabela 1 resume o sinal que cada regra lê no `model.bim` e se ela afirma pela presença ou pela ausência de algo. Só a MOD-003 afirma por ausência, e pode fazê-lo porque o terceiro teste se cumpre por inteiro: todos os relacionamentos de um modelo estão no próprio arquivo, de modo que uma tabela ausente de todos eles está, de fato, desconectada. Mesmo assim, como mostra o Capítulo 4, a regra erra em tabelas desconectadas de propósito, um caso que o critério não previu.

Tabela 1 – O sinal que cada regra lê e a forma da afirmação

Regra	**Sinal lido no `model.bim`**	**Afirma por**
MOD-001	Tabela de data gerada pelo produto (tempo automático)	presença
MOD-002	Relacionamento com filtro cruzado nos dois sentidos, exceto um-para-um	presença
MOD-003	Tabela ausente de todos os relacionamentos do modelo	ausência
MOD-005	`variations` da coluna de data da dimensão apontando para uma tabela automática	presença
MOD-006	Tabela no lado “um” de um relacionamento e no lado “muitos” de outro	presença
MOD-007	Relacionamento com cardinalidade “um” nas duas pontas	presença
PERF-001	Coluna calculada em DAX, exceto colunas de agrupamento e colunas de uma dimensão de data calculada em DAX	presença
PERF-003	Coluna de ponto flutuante com agregação padrão de soma	presença
DAX-001	Operador `/` com denominador não constante, por expressão	presença

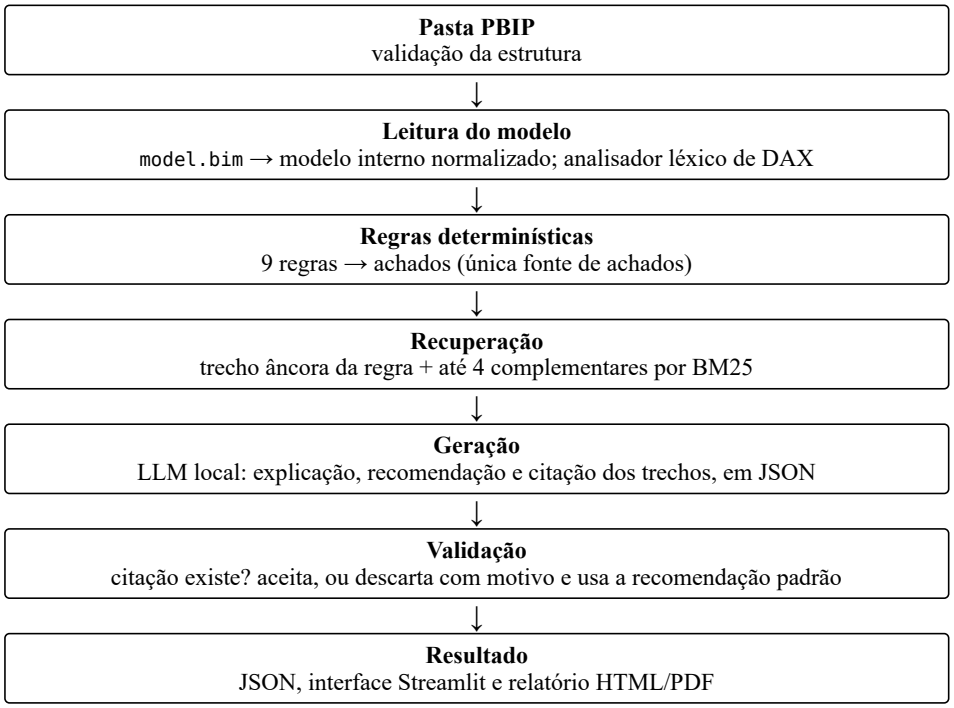
Fonte: Elaborado pelo autor a partir do código das regras (`core/rules/`).

As exceções da PERF-001 também vêm da documentação: as colunas de agrupamento têm o DAX escrito pela própria interface, e as páginas sobre tempo automático e tabelas de data recomendam criar a tabela de data com `CALENDAR` ou `CALENDARAUTO` e acrescentar colunas calculadas a ela (Microsoft, 2025). Sem essas exceções, a regra recomendaria desfazer o que a fonte recomenda fazer.

1.5 FLUXO DA AUDITORIA

A auditoria é uma função que executa as etapas em sequência, sem orquestração de agentes (ADR-003), conforme a Figura 2. O fluxo é linear e um arcabouço de agentes não traria ganho demonstrável; a sequência fixa também facilita explicar e testar cada etapa isoladamente.

Figura 2 – Fluxo da auditoria



Fonte: Elaborado pelo autor.

As regras são executadas em ordem estável e isoladas umas das outras: se uma regra falha com erro, ela não interrompe a auditoria, e sua saída parcial é descartada inteira, para que nenhum achado venha de uma execução incompleta. Antes de chamar o modelo de linguagem, a auditoria verifica que toda regra tem gabarito de âncora e que a seção âncora existe no índice; faltando qualquer uma, ela para em vez de gerar uma explicação sem fundamento. Além da linha de comando, a ferramenta tem uma interface em Streamlit e gera um relatório em HTML e PDF.

A Tabela 2 lista as tecnologias usadas (ADR-002). A escolha seguiu duas restrições do ambiente: tudo precisava instalar no Windows 11 sem compilador C++, e o modelo de linguagem precisava caber numa GPU de 6 GB. A recuperação foi escrita sem arcabouço, como um fluxo curto e próprio, porque um arcabouço como LangChain ou LlamaIndex acrescentaria abstrações que um fluxo linear não pede e que dificultariam explicar o funcionamento.

Tabela 2 – Pilha tecnológica da ferramenta

Camada	**Escolha**
Linguagem	Python 3.11, com Pydantic para o modelo interno validado
Leitura do modelo	`json` da biblioteca padrão sobre o `model.bim`; analisador léxico de DAX próprio
Recuperação	Fluxo próprio: coleta em HTML, extração por seção, índice BM25 (`rank-bm25`)
Modelo de linguagem	`qwen2.5:7b-instruct` quantizado em 4 bits, executado pelo Ollama na GPU local
Interface	Streamlit
Relatório	Jinja2 para o HTML; Playwright controlando o Microsoft Edge do sistema para o PDF
Testes e avaliação	pytest; pacotes próprios para o gabarito, a medição da recuperação e a rubrica da geração

Fonte: Elaborado pelo autor a partir da ADR-002.

Todo o desenvolvimento foi guiado por testes automatizados, escritos antes do código. Além dos testes de unidade, há testes que conferem artefatos contra a sua fonte: o catálogo de fontes da recuperação é regenerado e comparado com o arquivo versionado, a passagem citada por cada regra é procurada no texto extraído da base, e as tabelas de resultado desta monografia são geradas dos arquivos de resultado e comparadas com as que o texto inclui. Assim, um número que mude na origem faz a suíte falhar até que o documento seja atualizado.

1.6 BASE DE DOCUMENTAÇÃO E RECUPERAÇÃO

A base consultada pela recuperação é a documentação pública do Microsoft Learn. O plano inicial de clonar o repositório de origem da documentação não se sustentou, porque o repositório é privado. As páginas foram então coletadas em HTML a partir dos índices publicados de cada seção do site, depois de verificado que o arquivo `robots.txt` não bloqueia o caminho (ADR-004). O catálogo de fontes é gerado desses índices, e não digitado, e reúne 1.420 páginas: a seção de orientação do Power BI inteira, as referências completas de funções DAX e de Power Query M e as páginas âncora das regras. A cópia local das páginas fica fora do repositório; versionam-se o catálogo e o código de coleta. Seis artigos do SQLBI entram apenas como leitura complementar, com link, porque os termos do site não autorizam armazenar os artigos.

Cada página é extraída por seção, preservando a âncora e o caminho de títulos de cada uma, e as seções são os trechos indexados, com até 350 palavras. A recuperação tem dois níveis (ADR-002, emenda de 09/10/2026). O primeiro é fixo: a **seção âncora** da regra, a passagem transcrita quando a regra passou no segundo teste do critério, entra sempre como o trecho número 1. O segundo vem da busca, que acrescenta trechos complementares. Essa construção nasceu de uma falha observada no teste inicial do modelo de linguagem: com várias páginas no contexto, o modelo citou uma página válida, mas menos pertinente que a âncora. Com a âncora fixa, a passagem que sustenta o achado está sempre disponível, e a qualidade da busca passa a afetar só os complementares.

O método de busca foi escolhido por medição, com a régua definida antes de medir. Foram comparadas 24 combinações de fragmentação (por seção ou por janela de palavras), cabeçalho de contexto (com ou sem), tipo de busca (densa, com o modelo de vetores `bge-small-en-v1.5`; lexical, com BM25 (Robertson; Zaragoza, 2009); ou híbrida, por fusão recíproca de posições (Cormack; Clarke; Büttcher, 2009)) e corpus (completo ou só orientação). A regra de decisão, fixada antes, escolhia a maior taxa de acerto entre os cinco primeiros resultados no corpus completo e, em caso de empate próximo, a combinação mais simples. Venceu a

fragmentação por seção, sem cabeçalho, com BM25 sobre o corpus completo. Os números e as limitações dessa medição estão no Capítulo 4.

1.7 GERAÇÃO COM CITAÇÃO OBRIGATÓRIA

A explicação é redigida pelo `qwen2.5:7b-instruct` (Qwen Team, 2024), quantizado em 4 bits e executado localmente pelo Ollama numa GPU de 6 GB. O modelo foi escolhido num teste comparativo com a versão de 3 bilhões de parâmetros: os dois citaram corretamente nos cinco casos de teste, mas o menor errou um número que estava na própria evidência. A execução local tem uma razão prática, além do custo: o modelo semântico de um cliente não precisa sair da máquina de quem o audita. A janela de contexto foi fixada em 4.096 tokens, o máximo que cabe na memória da GPU sem descarregar camadas para a CPU, e a geração usa temperatura 0 e semente fixa.

O pedido ao modelo tem duas partes. A instrução diz que o problema já foi detectado por uma regra determinística e que o modelo não decide se ele existe; que deve usar só os trechos fornecidos; citar de 1 a 3 trechos pelo número; e responder em português do Brasil, de forma curta e acionável. A mensagem traz a regra, o objeto afetado, a evidência do achado e os trechos numerados, em inglês, a língua do corpus. A resposta tem de seguir um esquema JSON imposto pelo Ollama, com explicação, recomendação e a lista de citações. A Figura 3 reproduz a instrução de sistema, igual em todos os achados. A citação é pedida pelo número do trecho, e não pelo endereço da página, porque um modelo de 7 bilhões de parâmetros copia mal cadeias longas; o código traduz o número para página e seção.

Figura 3 – Instrução de sistema enviada ao modelo de linguagem

> Você é um auditor de modelos semânticos do Power BI.
>
> Uma regra determinística JÁ detectou o problema descrito pelo usuário. Você NÃO decide se o problema existe: sua tarefa é explicá-lo e recomendar a correção.
>
> Use somente as informações do achado e dos trechos de documentação numerados. Não invente números, nomes de objetos nem fatos.
>
> Responda em português do Brasil, em JSON, com:
> - “explicacao”: por que isso é um problema neste modelo;
> - “recomendacao”: o que fazer, de forma acionável;
> - “citacoes”: de 1 a 3 números dos trechos que sustentam a explicação.
>
> Explicação e recomendação juntas devem ter no máximo 120 palavras.

Fonte: Código da ferramenta (`core/analise/prompt.py`).

A validação é feita por código. Se a resposta não é JSON válido, se um campo vem vazio, se o número de citações sai do intervalo ou se algum número citado não existe no contexto, a resposta é descartada, sem nova tentativa, e o motivo é registrado. O achado continua no resultado com a recomendação padrão da regra e a marca “sem fundamentação gerada”: o descarte nunca apaga um achado. Uma resposta aceita passa ainda por uma conferência de superfície, que verifica se números e nomes de objetos citados na explicação aparecem no achado ou nos

trechos. Por ser uma heurística ainda não calibrada, ela registra divergências, mas não descarta. O Capítulo 4 mostra que essa conferência não mede o que a leitura humana chama de fidelidade.

As respostas podem ser guardadas num cache, indexado pelo pedido completo, pelo modelo e pelos parâmetros de geração. O cache torna a auditoria repetível em segundos com textos idênticos, o que é necessário porque a temperatura 0 não torna a geração do Ollama determinística: entre duas execuções, os totais se repetem, mas os textos e algumas citações variam. Ele vem desligado por padrão e, quando usado, é indicado no resultado e no relatório.

1.8 CONJUNTO DE AVALIAÇÃO

As métricas vêm de sete projetos públicos do repositório de exemplos do Power BI Desktop mantido pela Microsoft, sob licença MIT, convertidos manualmente para PBIP em TMSL (ADR-005). Eles cobrem sete domínios: vendas, despesas corporativas, recursos humanos, marketing, varejo, cadeia de suprimentos e funil comercial. Cinco vêm da revisão de 2026 dos exemplos e dois de amostras mais antigas, uma mistura que evita um conjunto uniformemente bom ou ruim. Por serem públicos, qualquer leitor pode refazer a avaliação com os mesmos arquivos. Esses projetos são chamados de P1 a P7.

Um oitavo projeto, P8, é um modelo de autoria própria sobre vendas de varejo online. Ele chegou a integrar o conjunto de métricas, mas saiu dele porque não pode ser reproduzido por terceiros, e passou a estudo de caso: mostra a ferramenta agindo em profundidade sobre um modelo com problemas reais, a começar pelo tempo automático ligado ao lado de uma dimensão de data própria. Nenhuma métrica do Capítulo 4 depende do P8.

O gabarito das regras foi anotado pelo autor. Para cada regra, um gerador independente da lógica da regra lista todos os objetos que poderiam violá-la — por exemplo, todas as tabelas, para a MOD-003, ou todas as colunas calculadas, para a PERF-001 —, acompanhados de fatos neutros sobre cada um. A anotação foi cega: a planilha não informava quais objetos a ferramenta marcara, e o anotador julgava cada um contra a pergunta da âncora, com os vereditos “viola”, “não viola” ou “indeterminado”, este último reservado aos casos em que o modelo não traz a informação necessária. Nenhuma regra mudou durante a anotação, e a versão medida foi congelada numa marca de versão do repositório no commit que a encerrou. O protocolo previa criar a marca antes de anotar; ela foi criada depois, mas sobre o mesmo código. Correções de regra feitas a partir daí são relatadas como análise de erros, medidas contra o mesmo gabarito, e não como resultado.

2 AVALIAÇÃO

A avaliação responde a três perguntas, uma para cada camada da ferramenta. As regras apontam os problemas certos? A recuperação encontra a passagem da documentação que sustenta cada achado? A explicação gerada pode ser entregue ao dono do modelo sem edição? Este capítulo apresenta o desenho da avaliação, os resultados de cada pergunta e o estudo de caso do projeto P8. Todas as tabelas de resultado foram geradas pelo código a partir dos arquivos de resultado versionados no repositório, que o leitor pode conferir.

2.1 DESENHO DA AVALIAÇÃO

Cada pergunta tem a sua medida, o seu gabarito e uma regra de decisão escrita antes da medição. Fixar a regra antes de medir impede que a escolha se ajuste ao resultado: quando o resultado contrariou a expectativa, a regra foi mantida e a divergência, declarada.

As proporções são apresentadas com o intervalo de confiança de 95% de Wilson (Wilson, 1927), mais adequado que o intervalo normal quando a amostra é pequena ou a proporção está perto de 0 ou de 1, situação frequente aqui: várias regras têm um a três achados, e várias proporções valem 1,00. Quando o anotador marcou um caso como indeterminado, a proporção é calculada sobre os casos decididos e acompanhada de dois limites, o **piso**, que conta os indeterminados contra a ferramenta, e o **teto**, que os conta a favor. A concordância entre o julgamento humano e os critérios automáticos é medida pela concordância simples e pelo kappa de Cohen (Cohen, 1960).

2.2 REGRAS

O gabarito das regras tem 350 linhas, uma por objeto candidato a violar uma regra nos projetos P1 a P7, anotadas às cegas, como descrito no Capítulo 3. A primeira medição levou a uma releitura das divergências, e 12 julgamentos foram revistos com base na documentação, cada um com o motivo registrado e o julgamento original preservado no histórico do repositório. A revisão andou nos dois sentidos. Sete linhas da MOD-005 passaram de “viola” a “indeterminado”, porque o `model.bim` não registra a marcação de tabela de data e não havia como afirmar que a tabela não estava marcada. Três linhas da DAX-001 passaram de “não viola” a “viola”, porque a página da âncora usa exatamente aquele padrão, uma divisão protegida por um teste de denominador diferente de zero, como exemplo do que deve ser trocado pela função `DIVIDE` (Microsoft, 2021). As demais revisões foram uma coluna gerada pelo recurso de agrupamento do Power BI e uma tabela de negócio sem nenhum relacionamento. A Tabela 3 mostra o resultado oficial, com as regras na versão congelada.

Tabela 3 – Desempenho das regras contra o gabarito em P1–P7 (resultado oficial (tag regras-gt))

Regra	VP	FP	FN	Indet.	Precisão	IC 95% (Wilson)	Revocação
DAX-001	6	0	0	0	1,00	[0,61; 1,00]	1,00
MOD-001	1	0	0	1	1,00	[0,21; 1,00]	1,00
MOD-002	0	0	0	0	—	—	—
MOD-003	1	6	0	0	0,14	[0,03; 0,51]	1,00
MOD-005	0	0	0	7	—	—	—
MOD-006	3	0	0	0	1,00	[0,44; 1,00]	1,00
MOD-007	1	0	0	0	1,00	[0,21; 1,00]	1,00
PERF-001	18	0	6	5	1,00	[0,82; 1,00]	0,75
PERF-003	16	0	0	0	1,00	[0,81; 1,00]	1,00
Total (micro)	46	6	6	13	0,88	[0,77; 0,95]	0,88

Fonte: Elaborado pelo autor a partir de `regras-2026-10-12.json`.

A precisão total foi de 0,88: dos 52 achados emitidos pelas regras, 46 foram confirmados pelo gabarito. A revocação total também foi de 0,88. Seis das nove regras tiveram precisão 1,00. Duas não emitiram achado algum: a MOD-002 porque nenhum dos sete projetos tem relacionamento bidirecional que ela cubra, e o gabarito confirmou a ausência, e a MOD-005 porque, com o tempo automático desligado, ela se cala por desenho, o falso negativo que o Capítulo 3 declarou; os sete casos que o anotador não pôde decidir aparecem na coluna de indeterminados. A amplitude dos intervalos de Wilson mostra o quanto o número de casos limita as conclusões por regra: a MOD-001 e a MOD-007 têm precisão 1,00 sobre um único achado, com intervalo de 0,21 a 1,00. As regras com mais casos, PERF-001 e PERF-003, têm intervalos estreitos, com piso acima de 0,80.

O projeto fixou, entre os seus riscos, um limiar de precisão de 0,7 por regra, abaixo do qual a regra seria revista. A referência industrial é mais exigente: as ferramentas de análise estática do Google só mantêm uma verificação se menos de 10% dos seus avisos forem falsos positivos efetivos, isto é, avisos sobre os quais o desenvolvedor não age (Winters; Manshreck; Wright, 2020). Só uma regra ficou abaixo do limiar: a MOD-003, com precisão de 0,14, um acerto e seis falsos positivos.

A análise desses seis falsos positivos mostrou duas causas. Quatro eram objetos que o Power BI cria desconectados de propósito, grupos de cálculo e parâmetros de campo, e que têm sinal estrutural próprio no `model.bim`: a propriedade `calculationGroup`, no primeiro caso, e uma anotação de metadados de parâmetro, no segundo. Os outros dois, `Logo` e `Indicators`, eram tabelas digitadas à mão para a apresentação do relatório, sem nenhum sinal que as distinga de uma dimensão digitada e esquecida. Na PERF-001, os seis falsos negativos tinham uma causa só: a exceção para colunas de tabelas de data estava larga demais e excluía também as tabelas de data importadas pelo Power Query, quando a documentação só recomenda colunas calculadas na tabela de data criada em DAX.

As duas regras foram corrigidas depois da medição oficial, sempre por sinal estrutural e nunca pelo nome do objeto, e medidas de novo contra o mesmo gabarito (Tabela 4). Esse resultado é **análise de erros**, não resultado: as correções foram motivadas pelos próprios casos em que são medidas, e a sua validação exigiria projetos que as regras ainda não viram.

Tabela 4 – Desempenho das regras após a correção da MOD-003 e da PERF-001 (análise de erros)

Regra	VP	FP	FN	Indet.	Precisão	IC 95% (Wilson)	Revocação
DAX-001	6	0	0	0	1,00	[0,61; 1,00]	1,00
MOD-001	1	0	0	1	1,00	[0,21; 1,00]	1,00
MOD-002	0	0	0	0	—	—	—
MOD-003	1	2	0	0	0,33	[0,06; 0,79]	1,00
MOD-005	0	0	0	7	—	—	—
MOD-006	3	0	0	0	1,00	[0,44; 1,00]	1,00
MOD-007	1	0	0	0	1,00	[0,21; 1,00]	1,00
PERF-001	24	2	0	5	0,92	[0,76; 0,98]	1,00
PERF-003	16	0	0	0	1,00	[0,81; 1,00]	1,00
Total (micro)	52	4	0	13	0,93	[0,83; 0,97]	1,00

Fonte: Elaborado pelo autor a partir de `regras-2026-10-12-pos-correcao.json`.

A MOD-003 subiu para 0,33 e continua abaixo do limiar, por causa das duas tabelas digitadas, uma limitação declarada da regra. A PERF-001 passou a ter revocação 1,00 e precisão 0,92. Parte desse ganho, porém, está em casos indeterminados que a regra passou a marcar: contados contra ela, a precisão da PERF-001 cai para 0,77.

O conjunto de avaliação também precisava de achados suficientes. O planejamento fixou um mínimo de cerca de 40 achados confirmados em P1 a P7, abaixo do qual um oitavo projeto seria acrescentado. Os 46 achados confirmados superam o mínimo, mas 16 deles vêm de uma única regra, a PERF-003; sem ela, seriam 30.

2.3 RECUPERAÇÃO

A recuperação foi avaliada em duas etapas. A primeira escolheu o método de busca; a segunda, feita depois que a âncora passou a entrar sempre no contexto, verificou se a busca entregava trechos complementares úteis.

Na primeira etapa, o gabarito de cada regra é a seção da documentação que contém a sua passagem âncora, identificada por uma frase característica. As consultas foram 62: uma por regra e uma por achado de P1 a P7, cada uma montada com os termos da regra e os dados do achado. Para cada consulta mediu-se se a seção âncora estava entre os K primeiros resultados (acerto@K), a fração de resultados pertinentes entre os K primeiros (precisão@K) e o inverso da posição do primeiro acerto (MRR). As médias são **macro**, calculadas primeiro por regra e depois entre as regras, para que a regra com mais achados não domine o resultado. A Tabela 5 mostra a combinação vencedora entre as 24 comparadas.

Tabela 5 – Recuperação vencedora (fragmento por seção, busca BM25, corpus completo): médias macro por regra em 62 consultas

K	Acerto@K	Precisão@K
1	0,44	0,44
3	0,72	0,24
5	1,00	0,20
10	1,00	0,10
MRR@10	0,61	—

Fonte: Elaborado pelo autor a partir de `recuperacao-2026-10-11.json`.

A seção âncora apareceu entre os cinco primeiros resultados em todas as regras, e na primeira posição em menos da metade delas. O número precisa de duas ressalvas, registradas na decisão. A primeira é um **efeito teto**: 7 das 12 combinações do corpus completo chegaram ao acerto@5 de 1,00, porque as consultas carregam os termos da própria regra, e a medida que decidia quase não separou as alternativas. A segunda é que uma única consulta decidiu o empate: a busca densa ficou fora dele só porque a consulta da MOD-005 encontrou a âncora na sétima posição, e essa regra, com uma consulta, pesa um nono da média. A ampliação do corpus para as referências completas de DAX e M, por outro lado, não prejudicou a recuperação: o corpus restrito à orientação não superou o completo.

A segunda etapa mudou a pergunta. Com a âncora fixa como primeiro trecho, a busca só escolhe os trechos 2 a 5, e a medida anterior deixou de informar. O autor julgou a pertinência das 89 seções distintas que as três buscas entregavam como complementares em 70 consultas, sem saber qual busca tinha entregado cada seção. A regra, fixada antes, só trocaria o BM25 por outra busca se ela superasse a média do BM25 em pelo menos 0,10 (Tabela 6).

Tabela 6 – Pertinência dos trechos complementares por estratégia de busca (média macro)

Estratégia	Principal	Piso	Teto
BM25	0,62	0,59	0,62
Densa (bge-small)	0,67	0,64	0,71
Híbrida	0,65	0,64	0,69

Fonte: Elaborado pelo autor a partir de `complementares-2026-10-12.json`.

Nenhuma alternativa superou o BM25 pela margem fixada, e ele foi mantido. A medição mostrou um problema que nenhuma troca de busca resolveria: na PERF-001, nenhuma das três buscas entregou complemento pertinente, porque a consulta dessa regra inclui a expressão DAX da coluna e atrai páginas de referência de funções. O defeito está na construção da consulta, não no tipo de busca. Registrou-se também que, considerando só as consultas por achado, que são as que a ferramenta de fato faz, a busca densa superaria o BM25 por 0,13; a regra aprovada antes da medição usava as 70 consultas e foi mantida.

2.4 GERAÇÃO

A geração foi avaliada sobre todos os 61 achados que as regras emitem em P1 a P7, sem amostragem. Como a geração depende só de quais achados existem, e não de como as regras os detectaram, foi usada a versão corrigida das regras; o resultado oficial da detecção continua sendo o da Tabela 3. Os 61 achados receberam explicação: nenhuma resposta foi descartada pela validação da citação. A rodada levou 8 minutos e 53 segundos e foi gravada no cache, de modo que repeti-la reproduz exatamente os textos avaliados.

Cada explicação foi julgada pelo autor numa planilha que não mostrava nenhum dos sinais automáticos, para que a comparação entre os dois fosse válida. Três itens foram julgados com “sim”, “não” ou “indeterminado”, com justificativa obrigatória no “não”:

- **pertinente**: pelo menos um trecho citado é a passagem da documentação que sustenta o achado, e não apenas um trecho existente no contexto;
- **fiel**: tudo o que a explicação afirma é sustentado pelos trechos citados ou pelos fatos do achado, sem números, nomes ou comportamentos inventados;
- **recomendação correta**: a recomendação resolve o problema neste modelo, sem erro técnico.

Uma explicação é **utilizável como está** quando recebe “sim” nos três itens. Não houve casos indeterminados. A Tabela 7 mostra o resultado.

Tabela 7 – Rubrica humana das explicações geradas: proporção de “sim” por item

Item	Sim / n	Proporção	IC 95% (Wilson)
Pertinente	61 / 61	1,00	[0,94; 1,00]
Fiel	32 / 61	0,52	[0,40; 0,64]
Recomendação correta	47 / 61	0,77	[0,65; 0,86]
Utilizável como está	23 / 61	0,38	[0,27; 0,50]

Fonte: Elaborado pelo autor a partir de `geracao-2026-10-14.json`.

A recuperação e a citação funcionaram: todas as 61 explicações citaram uma passagem pertinente. O problema está no que o modelo escreve a partir dela. Pouco mais da metade das explicações foi fiel aos trechos, e pouco mais de um terço pode ser entregue sem edição. A Tabela 8 mostra que os erros se concentram em duas regras.

Tabela 8 – Rubrica humana das explicações por regra

Regra	Achados	Fiel	Recomendação correta	Utilizável
DAX-001	6	1 / 6	6 / 6	1 / 6
MOD-001	1	0 / 1	0 / 1	0 / 1
MOD-003	3	3 / 3	0 / 3	0 / 3
MOD-006	3	2 / 3	2 / 3	1 / 3
MOD-007	1	1 / 1	0 / 1	0 / 1
PERF-001	31	9 / 31	24 / 31	6 / 31
PERF-003	16	16 / 16	15 / 16	15 / 16

Fonte: Elaborado pelo autor a partir de `geracao-2026-10-14.json`.

Os “não” seguem padrões, e não se distribuem ao acaso. Na PERF-001, que responde por metade dos achados, a maior parte das explicações infiéis repete a mesma distorção: o trecho diz que o VertiPaq **comprime** com menos eficiência as colunas calculadas, e a explicação diz que ele não as “compreende” eficientemente, ou fala em “descompressão”. Tudo indica um erro de tradução do modelo, que escreve em português sobre uma fonte em inglês. As recomendações erradas da PERF-001 propõem sintaxe DAX para o Power Query ou funções de M que não existem. Na DAX-001, cinco das seis explicações tratam como defeito o fato de `DIVIDE` devolver vazio, quando o trecho citado diz que esse é, em geral, o comportamento desejável para uma medida. Na MOD-003, as três recomendações mandam relacionar tabelas auxiliares que são desconectadas de propósito, o mesmo ponto cego da regra. Em contraste, a PERF-003 teve 15 de 16 explicações utilizáveis.

Por fim, a Tabela 9 compara o julgamento humano com os dois critérios automáticos da ferramenta: se a explicação cita a seção âncora e se a conferência de números e nomes não acusou divergência.

Tabela 9 – Concordância entre a leitura humana e a conferência automática (a: ambos sim; b: só o automático diz sim; c: só o humano diz sim; d: ambos não)

Item humano	Critério automático	a	b	c	d	Concordância	Kappa
Pertinente	Cita a seção âncora	60	0	1	0	0,98	0,00
Fiel	Sem aviso de números ou nomes	24	24	8	5	0,48	−0,08

Fonte: Elaborado pelo autor a partir de `geracao-2026-10-14.json`.

No item pertinente, a concordância foi de 0,98, mas o kappa foi zero. Não há contradição: quase todos os casos caíram na mesma célula, ambos “sim”, e o kappa, que desconta a concordância esperada ao acaso, perde o sentido quando uma categoria domina a tabela, um paradoxo conhecido da medida (Feinstein; Cicchetti, 1990). O que se pode afirmar é mais simples: a combinação da âncora fixa com a citação obrigatória entregou evidência pertinente em todos os achados, e o único caso em que o modelo não citou a âncora citou outra passagem que também sustentava o achado.

No item fiel, a conferência automática concordou com o humano em menos da metade dos casos, com kappa de −0,08, nenhum acordo além do acaso. Em 24 explicações a conferência não viu problema e a leitura humana encontrou infidelidade. A razão é que as infidelidades reais são de sentido, como “compreende” no lugar de “comprime”, e não números ou nomes trocados, que é o que a conferência verifica. O resultado mudou a ferramenta: o aviso, que se chamava “divergência de fidelidade”, passou a se chamar “números ou nomes sem origem nos trechos”, e toda explicação gerada passou a vir com uma nota dizendo que a ausência de aviso não garante a correção do texto.

2.5 ESTUDO DE CASO: O PROJETO P8

O P8 é um modelo de vendas de varejo online, de autoria própria, com 19 tabelas, 93 medidas e 35 colunas calculadas. Ele foi o primeiro projeto auditado e o que motivou várias decisões de método, mas não entra nas métricas, porque não pode ser reproduzido por terceiros. A ferramenta encontra nele 15 achados de oito regras. O mais característico é o tempo automático ligado: o modelo tem uma dimensão de data própria e, ao lado dela, quatro tabelas de data geradas pelo Power BI, uma de modelo e três locais, uma para cada coluna de data. A MOD-001 aponta as quatro, e a MOD-005 aponta a dimensão própria, que, pela presença dessas tabelas, comprovadamente não está marcada como tabela de data.

Figura 4 – Achado do P8 com a explicação gerada pela ferramenta (identificador da tabela abreviado)

Achado (regra MOD-001): A tabela oculta ‘LocalDateTable_0d3564f1-…’ foi gerada pelo Tempo automático de data/hora, não pelo autor do modelo.

Explicação: A tabela ‘LocalDateTable_0d3564f1-…’ foi gerada pelo Auto date/time, o que pode aumentar o tamanho do modelo e dificultar a propagação de filtros entre tabelas. Isso não é recomendado para modelos mais complexos ou quando uma única tabela de datas é desejável.

Recomendação: Crie uma tabela de datas manualmente usando DAX, como a função CALENDAR ou CALENDARAUTO, e conecte-a às tabelas relevantes. Isso permitirá uma propagação de filtros mais eficiente e manterá o modelo mais limpo.

Trechos citados: [1] Auto date/time guidance in Power BI Desktop › Recommendations (âncora); [3] … › Considerations; [5] Design guidance for date tables in Power BI Desktop › Use Auto date/time.

Fonte: Saída da ferramenta no projeto P8, execução de 13/10/2026.

A Figura 4 mostra um desses achados como sai da ferramenta. A explicação é correta e está ancorada: o tamanho do modelo e a impossibilidade de propagar um filtro de data a várias tabelas estão entre as considerações da página citada, e a recomendação é a que a própria página faz (Microsoft, 2025). Na primeira execução completa, todos os 15 achados receberam explicação com citação válida, e 14 citaram a seção âncora da regra.

A leitura à mão das 15 explicações, porém, encontrou dois erros que nenhum critério automático acusou, ambos na MOD-006. Num deles, a recomendação mandava integrar numa única tabela a dimensão de produto, a de subcategoria e também a tabela de fatos de vendas; juntar fatos e dimensão é um erro de modelagem. No outro, recomendava uma tabela “normalizada” quando a fonte diz o contrário, desnormalizar. Como todos os nomes citados existiam no achado, a conferência automática não tinha o que acusar. Foram esses casos que levaram a rubrica a ter um item próprio para a correção da recomendação.

Quanto ao desempenho, a geração custou em média 8,1 segundos por achado, com pior caso de 10,5 segundos, numa GPU de 6 GB, e a auditoria completa do P8 levou 123 segundos, incluindo a carga do modelo. Duas execuções do mesmo projeto não produzem textos idênticos, apesar da temperatura 0: os totais se repetem, mas a redação e algumas citações variam, e numa das execuções pela interface 13 dos 15 achados citaram a âncora. Com o cache ligado, a segunda auditoria do P8 caiu de 165,6 segundos para 1,5 segundo, com textos idênticos aos da primeira.

REFERÊNCIAS

COHEN, Jacob. A coefficient of agreement for nominal scales. **Educational and Psychological Measurement**, v. 20, n. 1, p. 37–46, 1960.

CORMACK, Gordon V.; CLARKE, Charles L. A.; BÜTTCHER, Stefan. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. **Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '09), Boston**, p. 758–759, 2009.

FEINSTEIN, Alvan R.; CICCHETTI, Domenic V. High agreement but low kappa: I. The problems of two paradoxes. **Journal of Clinical Epidemiology**, v. 43, n. 6, p. 543–549, 1990.

MICROSOFT. **DIVIDE function vs divide operator (/) in DAX**. 2021. Disponível em: <https://learn.microsoft.com/en-us/dax/best-practices/dax-divide-function-operator>. Acesso em: 8 out. 2026.

MICROSOFT. **Data reduction techniques for Import modeling**. 2024a. Disponível em: <https://learn.microsoft.com/en-us/power-bi/guidance/import-modeling-data-reduction>. Acesso em: 15 out. 2026.

MICROSOFT. **Understand star schema and the importance for Power BI**. 2024b. Disponível em: <https://learn.microsoft.com/en-us/power-bi/guidance/star-schema>. Acesso em: 15 out. 2026.

MICROSOFT. **Auto date/time guidance in Power BI Desktop**. 2025. Disponível em: <https://learn.microsoft.com/en-us/power-bi/guidance/auto-date-time>. Acesso em: 15 out. 2026.

MICROSOFT. **Power BI Desktop project semantic model folder**. 2026. Disponível em: <https://learn.microsoft.com/en-us/power-bi/developer/projects/projects-dataset>. Acesso em: 15 out. 2026.

QWEN TEAM. Qwen2.5 Technical Report. **arXiv preprint arXiv:2412.15115**, 2024.

ROBERTSON, Stephen; ZARAGOZA, Hugo. The Probabilistic Relevance Framework: BM25 and Beyond. **Foundations and Trends in Information Retrieval**, v. 3, n. 4, p. 333–389, 2009.

WILSON, Edwin B. Probable inference, the law of succession, and statistical inference. **Journal of the American Statistical Association**, v. 22, n. 158, p. 209–212, 1927.

WINTERS, Titus; MANSHRECK, Tom; WRIGHT, Hyrum. **Software Engineering at Google: Lessons Learned from Programming Over Time**. Sebastopol: O'Reilly Media, 2020.